

Assignment 4: LLM Reinforcement Learning

GR-REINFORCE and GRPO: Implementation and Experimental Analysis

1. Approximate KL Regularization

Let a be a token sampled from the current policy $\pi_{\theta}(\cdot|s)$, and define $\Delta(a) = \log \pi_{\text{ref}}(a|s) - \log \pi_{\theta}(a|s)$. The sampled-token estimator is

$$k(a) = \exp(\Delta(a)) - \Delta(a) - 1.$$

Since a is sampled from $\pi_{\theta}(\cdot|s)$,

$$\mathbb{E}_{a \sim \pi_{\theta}}[\exp(\Delta(a))] = \sum_a \pi_{\theta}(a|s) [\pi_{\text{ref}}(a|s) / \pi_{\theta}(a|s)] = \sum_a \pi_{\text{ref}}(a|s) = 1.$$

Therefore, $\mathbb{E}[k(a)] = -\mathbb{E}[\Delta(a)] = \mathbb{E}[\log \pi_{\theta}(a|s) - \log \pi_{\text{ref}}(a|s)] = \text{KL}(\pi_{\theta}(\cdot|s) \parallel \pi_{\text{ref}}(\cdot|s))$. Thus $k(a)$ has the correct KL expectation. It is also nonnegative because $\exp(x) - x - 1 \geq 0$ for every x , with equality only at $x = 0$. Unlike the signed estimator $-\Delta(a)$, each sampled-token contribution therefore behaves as a nonnegative divergence penalty.

Computing exact KL at every generated-token position would require the complete vocabulary distribution from both the trainable and reference policies. For a large vocabulary and long completions, retaining and reducing these distributions is expensive in both compute and memory. The sampled-token estimator uses only the log-probabilities of generated tokens and can be averaged with the same completion mask used by the policy objective.

2. Implementation

I implemented the required components in the order of the training data flow: per-token log-probabilities, completion masking, approximate KL computation, minibatch iteration, group-relative advantages, GR-REINFORCE, and GRPO. The main indexing detail is next-token alignment: logits at position $t-1$ score token x_t . Prompt tokens and padding are removed by the completion mask so that the policy loss and KL term are evaluated only on generated tokens.

For GR-REINFORCE, completion-token log-probabilities are averaged within each sequence and weighted by the scalar group-relative advantage. The rollout is used once, preserving a strictly on-policy update. GRPO instead caches rollout-policy log-probabilities, forms token-level importance ratios, and applies PPO clipping. This permits multiple optimization epochs over the same rollout while constraining large deviations from the behavior policy.

One confusion point was the distinction between sequence-level advantages and token-level importance ratios. The group-relative advantage is computed once for each completion and broadcast across its valid tokens, whereas the GRPO ratio and clipping decision are computed independently at each generated-token position.

3. Format Copy Results

The format-copy task was used as an end-to-end validation of the RL pipeline. Both GR-REINFORCE and GRPO successfully learned the task. At the final evaluation near step 50, both runs reached 1.00 exact integer match. The answer-tag metric and the strict-XML-only metric also reached 1.00. The policies therefore learned both the target value and the required output structure.

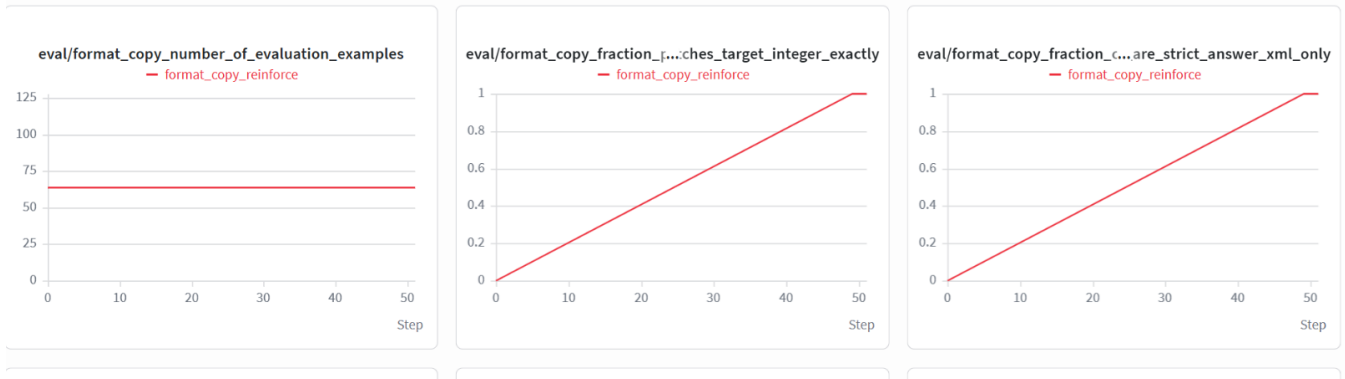


Figure 1. Format Copy + GR-REINFORCE evaluation. Final exact-match and XML-format metrics reach 1.00.

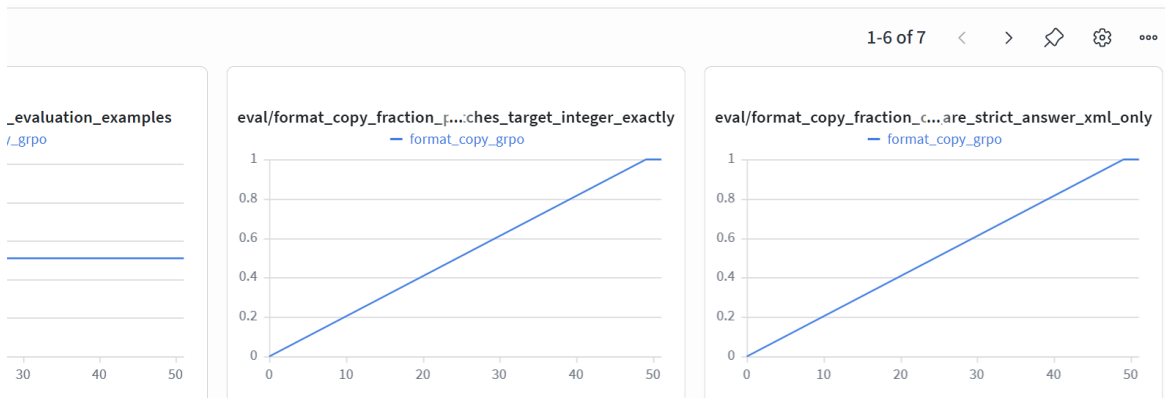


Figure 2. Format Copy + GRPO evaluation. Final exact-match and XML-format metrics reach 1.00.

4. GR-REINFORCE vs. GRPO on Math Hard

The math-hard commands make the first 200 iterations a controlled algorithmic comparison. The rollout batch size, group size, minibatch size, gradient accumulation, learning rate, KL coefficient, sampling temperature, and generation limits are matched. The main difference is sample reuse: GR-REINFORCE uses each rollout for one on-policy update, while GRPO reuses it for two PPO epochs.

This comparison is informative because faster improvement by GRPO over an equal number of rollout iterations indicates more optimization progress per collected sample. Importance ratios account for changes between the rollout policy and current policy, and PPO clipping limits the effect of tokens whose ratios move too far. GR-REINFORCE avoids this off-policy correction but extracts only one optimizer update from each rollout.

Rollout reward can fluctuate considerably because each iteration contains a limited set of sampled prompts and completions. The held-out boxed-answer exact-match metric is therefore the preferred curve for comparing the two methods. Approximate KL provides a complementary measure of policy drift, and GRPO clip fraction indicates how frequently reuse pushes token ratios outside the allowed PPO interval.

5. GRPO Hyperparameter Study on Format Copy

The ablation study examines the stability-sample-efficiency trade-off produced by PPO epochs, KL regularization, clipping, and optimizer-step frequency. These parameters interact because each controls how much the current policy can change while reusing one rollout.

Hyperparameter change	Effect on optimization	Diagnostic interpretation
Increase PPO epochs	More reuse per rollout	Can improve sample efficiency; excessive reuse increases policy mismatch and clipping.
Decrease KL coefficient	Weaker reference constraint	Allows faster policy movement but may increase approximate KL and instability.
Increase KL coefficient	Stronger reference constraint	Can stabilize updates but may suppress reward-driven learning.
Decrease clip epsilon	More conservative PPO updates	More ratios may be clipped, restricting otherwise useful updates.
Increase clip epsilon	Larger permitted ratio changes	Can accelerate learning but weakens protection against large policy shifts.
grad_accum_steps = 1	More optimizer steps per rollout	Can speed adaptation but may amplify KL growth and clip fraction.

The default setting balances reuse and stability: two PPO epochs provide additional optimization from each rollout, KL regularization limits drift from the frozen reference, and clipping constrains unusually large probability-ratio changes. The grad_accum_steps=1 configuration is especially aggressive because the optimizer updates after every minibatch rather than after accumulating gradients across the rollout, making KL and clip fraction important stability diagnostics.

6. Qualitative Behavior on Math Hard

The math-hard reward separates answer correctness from output-format compliance. The dominant reward is assigned when the boxed-answer parser extracts the correct number. Smaller auxiliary terms reward the required boxed structure and cases in which the relaxed last-number parser recovers the correct value even though the boxed parser fails. These auxiliary signals facilitate exploration before the model consistently solves the complete task.

An informative failure case is a completion with the correct boxed structure but an incorrect numerical value. This shows that the model can learn the surface form of the reward before mastering the mathematics. A contrasting successful completion contains both the correct value and the required boxed format. Together, these behaviors illustrate why auxiliary formatting rewards can support learning while exact mathematical correctness remains the primary objective.

7. Conclusion

This assignment demonstrates the practical difference between strictly on-policy GR-REINFORCE and PPO-style GRPO in LLM post-training. Group-relative advantages provide a prompt-local learning signal without a learned value function, while the sampled-token KL estimator regularizes the policy without a full-vocabulary KL computation. The format-copy experiments validate both implementations, with each method reaching 1.00 on the final task and formatting metrics. GRPO can extract more optimization from each rollout through clipped sample reuse, but the ablation analysis shows that this gain must be balanced against policy drift through PPO epochs, KL regularization, clipping, and optimizer-step frequency.